

Стандарт разработки на СБиС++

Отступы

1. Величина отступа (табуляции) – **3 (три)** знакоместа, то есть пробела. Причём ставятся именно пробелы, а не символ табуляции.
2. Каждое выражение должно начинаться с новой строки.
3. Между логическими блоками кода рекомендуется ставить пустую строку.

Фигурные скобки

Вариант 1

Открывающая фигурная скобка (символ {) ставится на новой строке без отступа. Текст после скобки начинается на следующей строке с отступом. Закрывающая фигурная скобка (символ }) ставится ровно под открывающей.

Например:

```
1  Если( пА > пБ )
2  {
3      пА = пБ;
4  }
```

Вариант 2

Обе фигурные скобки ставятся на одной строке.

```
1  <!-- Хорошо -->
2  Если( пА > пБ ) { пА = пБ; пБ = пЦ; }
```

```
1  <!-- Плохо. Недопустимые варианты -->
2  Если(пА > пБ)
3  { пА = пБ; пБ = пЦ; }
4
5  Функция Фу()
6  { пА = пБ; }
7
8  Функция ФуФу()
9  {
10     пА = пБ;
11     пБ = пЦ;
12 }
13
14 Пока(пА < пБ) {
15     пА++;
16 }
```

Круглые скобки

1. Круглые скобки после функций и операторов «ДляВсех», «Если» и тому подобное ставятся без пробела.
2. Внутри скобок переменные или выражения отделяются пробелами.

3. Возвращаемое значение не нужно заключать в скобки.

Пример:

```
1 Если(пА > пБ)
2     ДляВсех(Наименований)
3         Проводка(Д70, К50, Сумма);
4 Вернуть Сумма;
```

Выражения

Присваивание **всегда** обрамляется с обеих сторон пробелами. Каждое выражение должно начинаться с новой строки.

Например:

```
1 Переб пА = пБ + пЦ;
2 пЦ = пБ - пФ;
```

При использовании оператора «?:» обязательно должны быть поставлены пробелы:

```
1 пА = пА < пБ ? пА : пБ;
```

Не допускается использование оператора «?:» как замены для «если»:

```
1 <!-- Плохо -->
2 пБ < пА ? пА = 10 : пБ = 20;
```

Имена переменных и функций

Принято следующее соглашение по именованию:

Переменная	пИмяДляПримера
Массивы	мИмяДляПримера
Объекты	оИмяДляПримера
Функции	ИмяДляПримера()
Константы	ИМЯ_ДЛЯ_ПРИМЕРА
Поля таблиц базы данных	ИмяДляПримера

- Имя переменной обязательно должно начинаться с «п», «м», «о».
- Буква «ё» в именах не должна использоваться.
- Не использовать имена переменных, совпадающих с именами полей, параметров или функций, кроме как к обращению к этому полю или параметру.
- Имена переменных должны быть содержательными, например, **пА** – ни о чем не говорит, **пОтчетныйПериод** – говорит о многом.
- Не рекомендуется изменять значения полей, если эти изменения не требуется сохранять в базе данных. То есть не следует указывать в левой части оператора присваивания имя поля или выборки.

```
1 <!-- Хорошо -->
2 пСумма = Сумма * Коэф;
3 <field>пСумма</field>
```

```
1 <!-- Плохо -->
2 Сумма *= Коэф;
```

Условия

При написании условий лучше "думать" позитивно:

```
1 <!-- Хорошо -->
2 Если( а == 1 )
3     Действие1;
4 Иначе
5     Действие2;

1 <!-- Плохо -->
2 Если( а != 1 )
3     Действие2;
4 Иначе
5     Действие1;
```

Функции

Создание

Для создания функции можно воспользоваться шаблоном. Чтобы его открыть нужно:

1. Нажать <Ctrl + I> в редакторе (калькуляторе);
2. Выбрать раздел «Операторы языка»;
3. Выбрать оператор «Функция».

В редактор будет вставлена такая конструкция:

```
1  Функция
2  # Описание :
3  #
4  # Синтаксис :
5  #
6  # Параметры :
7  #
8  # Возвращает :
9  #
10 # Автор :
11 {
12 }
```

Описание

1. Название пользовательской функции не должно совпадать с названием встроенной функции, даже если у нее другое количество параметров.
2. После названия функции перед телом должен быть подробный комментарий функции, как в справочном руководстве, указав:
 - **Описание** – краткое описание что делает и возможно где используется;
 - **Синтаксис** – указывается только для функций с переменным числом параметров; это шаблон с указанием имен возможных параметров, необязательные параметры указываются в квадратных скобках;
 - **Параметры** – тип и назначение параметров, в том числе контекстного объекта, указываемого перед точкой;
 - **Возвращает** – тип и возможные возвращаемые значения, если ничего не возвращает, то описание результатов работы;

- **Автор** – Фамилия сотрудника, создавшего функцию. И возможно соавтор, который существенно модифицировал ее.

```

1  Функция оТовар.ТекстСтавкаНДС( пТипНДС, ... )
2  # Описание:
3  #   Определяет ставку НДС для товара на дату
4  #   в зависимости от того если или нет НДС в данной ситуации
5  # Синтаксис:
6  #   оТовар.ТекстСтавкаНДС( пТипНДС[, пДата ] )
7  # Параметры:
8  #   пТипНДС (число) – тип НДС ( 1 – есть НДС, 0 – нет НДС ),
9  #                               обычно определяется флагом НДС в документе
10 #   пДата (дата)      – дата на которую определяется ставка НДС
11 #                               если не указана, то берется текущая дата
12 #   оТовар (объект) – товар, запись из таблицы складская картотека
13 #                               или номенклатура
14 # Возвращает:
15 #   (текст) – ставка НДС
16 #                               если пТипНДС – 0 (без НДС), то возвращает «ОСВ»
17 # Автор: Макаров
18 {
19     ...
20 }
```

3. В функции всегда явно определять все используемые локальные переменные, чтобы случайно не изменить существующие глобальные переменные, например:

```

1  перемен пПеремен1, пПеремен2;
```

или так:

```

1  перемен пПеремен1 = 1;
2  ...
3  перемен пПеремен2 = 2;
```

4. Описывать переменные желательно около места их первого использования, а не все в самом начале.
5. Если переменная используется только внутри блока или цикла и после этого блока она больше не используется, то определять эту переменную нужно внутри этого блока.
6. При вызове функций без параметров всегда указывать круглые скобки:

```

1  пСуммаДок = СуммаДок()
```

7. Запрещается использовать в функции объекты и переменные, которые явно в нее не передаются. Все используемые объекты и переменные должны передаваться в качестве параметров функции или в качестве контекста.

```

1  <!-- Плохо. Делать так запрещено -->
2  Функция СчетСклада()
3  {
4      Вернуть Товар.СЧ_СКЛАДА;
5  }

1  <!-- Хорошо -->
2  Функция оТовар.СчетСклада()
3  {
4      Вернуть оТовар.СЧ_СКЛАДА;
5  }

1  <!-- Хорошо -->
2  Функция СчетСклада( оТовар )
```

```

3  {
4      Вернуть оТовар.СЧ_СКЛАДА;
5  }

```

8. При написании оператора *Вернуть* возвращаемое выражение не нужно заключать в скобки.
9. Функция не должна определять глобальные переменные или массивы, если требуется, то нужно вернуть массив или передать туда эти переменные, используя оператор **Перем**.
10. При создании внешних обработчиков необходимо после окончания разработки функционала удалять из них лишние (пустые) функции.
11. Если вы вносите изменения в функцию, автором которой вы не являетесь, то у измененного кода нужно ставить комментарий «Изменил: Фамилия», если целиком переписываете функцию, то ставьте себя соавтором в описании функции.

Комментарии

1. Комментарии **очень** желательны.
2. При использовании функции Лицо(«Тема»), обязательно ставить комментарий с названием этого лица.

```

1  оСтатья = Лицо( «СТРА», «Статьи» ); # СТРА – Страхование грузов

```

Рекомендации по стилю программирования

Не должно быть дублирования кода.

```

1  <!-- Хорошо -->
2  Если( Найти( Вверх( Наим.Наименование), "СТРАХОВ" ) )
3      перем оЛицо = Лицо( "СТРА", "Ан" );
4  Иначе
5      перем оЛицо = Лицо( "ТРАН", "Ан" );Проводка( Д44_1, оЛицо, К60_1,
Лицо1, СуммаСебест );

1  <!-- Плохо -->
2  Если( Найти( Вверх( Наим.Наименование), "СТРАХОВ" ) )
3      Проводка( Д44_1, Лицо( "СТРА", "Ан" ), К60_1, Лицо1, СуммаСебест );
4  Иначе
5      Проводка( Д44_1, Лицо( "ТРАН", "Ан" ), К60_1, Лицо1, СуммаСебест );

```